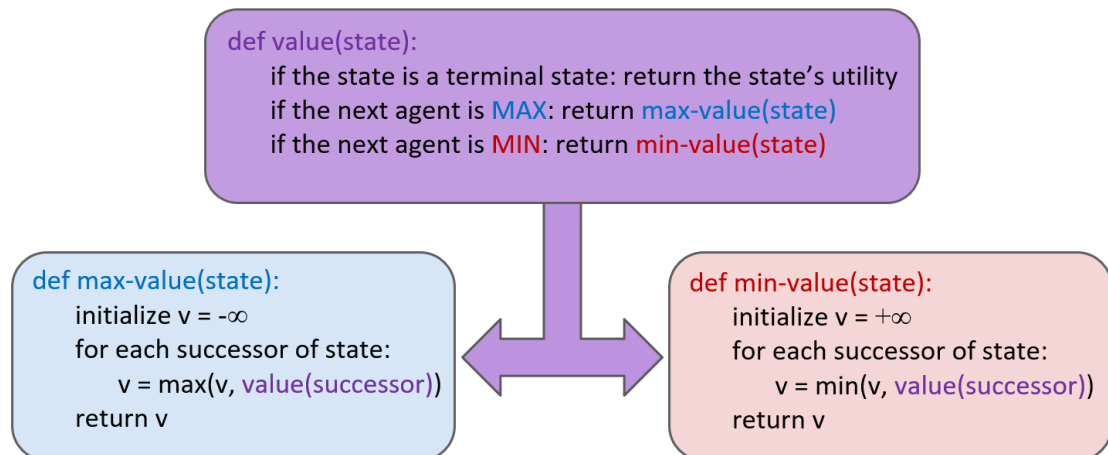


1. Minimax

打开 multiAgents.py 文件，补全其中 MinimaxAgent 类的 max_value 和 min_value 函数，其中 Minimax 算法的伪代码为：

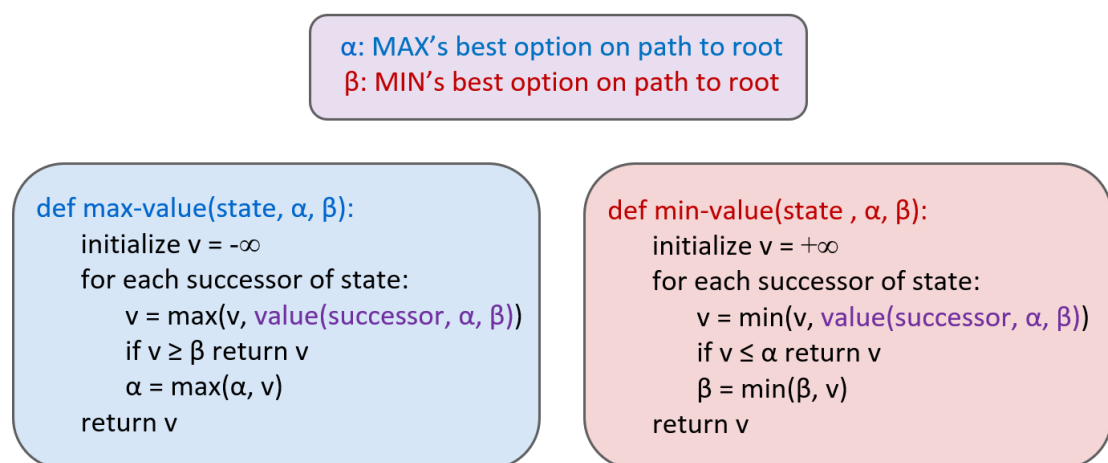


测试时在 Parameters 处写入：-p MinimaxAgent -l minimaxClassic -a depth=4

注意：深度为 4 的 Minimax 搜索并不能保证获胜，但大致胜率为 60%，可以在 Parameters 后加上 -n 10 来测试连续 10 轮的结果。

2. Alpha-Beta 剪枝

打开 multiAgents.py 文件，补全其中 AlphaBetaAgent 类的 max_value 和 min_value 函数，其中 Alpha-Beta 剪枝算法的伪代码为：

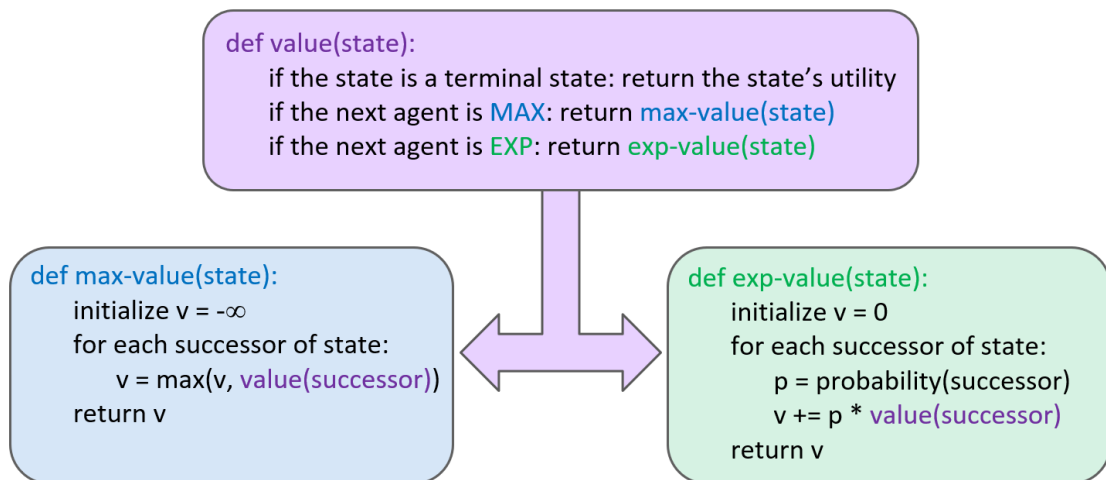


测试时在 Parameters 处写入：-p AlphaBetaAgent -a depth=3 -l smallClassic

此处可以对比使用 MinimaxAgent 和 AlphaBetaAgent 在深度同为 3 时的运行速度

3. Expectimax

打开 multiAgents.py 文件，补全其中 ExpectimaxAgent 类的 max_value 和 min_value 函数，其中 Expectimax 算法的伪代码为：



测试时在 Parameters 处写入： -p ExpectimaxAgent -l minimaxClassic -a depth=3， 同时也可以使用以下 Parameters 对比 Minimax 和 Expectimax 在随机 ghost 下的表现：

- 1) -p AlphaBetaAgent -l trappedClassic -a depth=3 -n 10
- 2) -p ExpectimaxAgent -l trappedClassic -a depth=3 -n 10